

Technical Exploration of I/O, Asynchronous Programming, Interrupts, and Memory

Dr. Probir Roy

February 17, 2025

Agenda

- ▶ Introduction to Computer Organization
- ▶ Detailed Analysis of I/O
- ▶ Asynchronous Programming Concepts
- ▶ Interrupt Mechanisms
- ▶ Memory Architecture and Management
- ▶ Integrated System Case Study

Introduction to Computer Organization

- ▶ Modern computer systems are built from a few fundamental components: CPU, Memory, and I/O.
- ▶ Understanding these underlying principles is crucial for designing and troubleshooting embedded and real-time systems.
- ▶ This presentation covers both theoretical and practical aspects.

Fundamental Components

- ▶ **Input/Output (I/O)** – Data exchange with the external world.
- ▶ **Asynchronous Programming** – Concurrent task execution.
- ▶ **Interrupts** – Immediate response to high-priority events.
- ▶ **Memory** – Data and instruction storage.

I/O: Definition and Significance

- ▶ **Definition:** Mechanisms for exchanging data between the computer and external devices.
- ▶ **Significance:**
 - ▶ Enables sensor data acquisition.
 - ▶ Supports user interaction and actuator control.

I/O: Classification

- ▶ **Input Devices:** Sensors, keyboards, cameras.
- ▶ **Output Devices:** LEDs, displays, motors.
- ▶ **Bidirectional Devices:** Touchscreens, communication transceivers.

I/O: Communication Protocols

- ▶ Common protocols: SPI, I²C, UART, CAN.
- ▶ Critical aspects: Signal integrity, voltage level translation, error checking.
- ▶ These protocols standardize communication between devices.

I/O: Hardware Perspective

- ▶ Direct register-level access to I/O peripherals.
- ▶ **Memory-mapped I/O** vs. **Port-mapped I/O**.
- ▶ **Diagram:** CPU $\leftrightarrow$ Bus $\leftrightarrow$ I/O Devices.

I/O: Software Abstraction

- ▶ Device drivers and APIs abstract the hardware details.
- ▶ Example: Arduino's `digitalWrite()` function hides register-level operations.
- ▶ Enhances portability and simplifies development.

I/O: Practical Example - LED Control

- ▶ Scenario: Toggling a built-in LED on the Arduino Nano BLE 33.
- ▶ Objective: Illustrate direct hardware control via I/O registers.

I/O: LED Control Code Walk-through

```
1  const int ledPin = LED_BUILTIN;
2
3  void setup() {
4      pinMode(ledPin, OUTPUT);
5      Serial.begin(9600);
6      while (!Serial);
7      Serial.println("I/O Example: LED control initiated")
          ;
8  }
9
10 void loop() {
11     digitalWrite(ledPin, HIGH);
12     Serial.println("LED ON");
13     delay(500);
14     digitalWrite(ledPin, LOW);
15     Serial.println("LED OFF");
16     delay(500);
17 }
```

Asynchronous Programming: Introduction

- ▶ **Definition:** Execution model that allows multiple tasks to run concurrently.
- ▶ Contrast with synchronous (sequential) execution.
- ▶ Key for reducing latency and improving responsiveness.

Asynchronous Programming: Theoretical Foundations

- ▶ Event-driven programming and non-blocking I/O.
- ▶ Use of callbacks, event loops, and futures/promises.
- ▶ Alternative models: Multi-threading vs. event-driven designs.

Asynchronous Programming: Benefits

- ▶ Reduced latency and enhanced system responsiveness.
- ▶ Improved resource utilization by overlapping I/O with computation.
- ▶ Suitable for real-time applications and high-performance systems.

Asynchronous Programming: Applications

- ▶ Real-time sensor monitoring.
- ▶ Responsive user interfaces in mobile and web applications.
- ▶ Network communications and multimedia streaming.

Asynchronous Programming: Timer Interrupt Example

- ▶ Use a timer interrupt for periodic tasks.
- ▶ Relevant for embedded systems such as the Arduino Nano BLE 33.

Asynchronous Programming: Code Example

```
1  #include "mbed.h"
2
3  Ticker ticker;
4  volatile bool ledState = false;
5
6  void toggleLED() {
7      ledState = !ledState;
8      digitalWrite(LED_BUILTIN, ledState);
9  }
10
11 void setup() {
12     pinMode(LED_BUILTIN, OUTPUT);
13     Serial.begin(9600);
14     while (!Serial);
15     Serial.println("Asynchronous Timer Interrupt Example
16                     Started");
17     ticker.attach(toggleLED, 1.0); // Trigger every 1
18                                     second
19 }
20
21 void loop() {
22     Serial.print("LED is ");
```

Asynchronous Programming: Best Practices

- ▶ Keep Interrupt Service Routines (ISRs) short and non-blocking.
- ▶ Use the `volatile` qualifier for shared variables.
- ▶ Beware of potential pitfalls such as race conditions and priority inversion.

Interrupts: Introduction and Definition

- ▶ **Definition:** Interrupts force the CPU to suspend current tasks to handle higher-priority events.
- ▶ Critical for real-time and safety-critical systems.

Interrupts: Types and Examples

- ▶ Hardware interrupts: External signals from peripherals.
- ▶ Software interrupts: Generated by software conditions or system calls.
- ▶ Exceptions: Faults that occur during execution.

Interrupts: System Architecture

- ▶ **Interrupt Controller** and **Interrupt Vector Table**.
- ▶ Diagram: Flow of an interrupt signal from the device to the CPU and back.
- ▶ Explanation of context switching and state saving.

Interrupts: Design Considerations

- ▶ Interrupt latency and response time.
- ▶ Deterministic behavior for safety-critical applications.
- ▶ Guidelines for managing nested interrupts and priorities.

Interrupts: Practical Example

```
1 // Same timer-based interrupt example as before
2 #include "mbed.h"
3
4 Ticker ticker;
5 volatile bool ledState = false;
6
7 void toggleLED() {
8     ledState = !ledState;
9     digitalWrite(LED_BUILTIN, ledState);
10 }
11
12 void setup() {
13     pinMode(LED_BUILTIN, OUTPUT);
14     Serial.begin(9600);
15     while (!Serial);
16     ticker.attach(toggleLED, 1.0);
17 }
18
19 void loop() {
20     Serial.print("LED is ");
21     Serial.println(ledState ? "ON" : "OFF");
22     delay(1000);
```

Interrupts: Real-World Applications

- ▶ Automotive systems: ABS, airbag deployment.
- ▶ Medical devices: Heart rate monitors, pacemakers.
- ▶ Industrial control: Process automation, safety systems.

Memory: Introduction and Importance

- ▶ **Definition:** Storage systems for data and instructions.
- ▶ Critical for performance, resource management, and system stability.

Memory: Memory Hierarchy

- ▶ Levels: Registers, Cache, RAM (Volatile), and Non-volatile Memory (Persistent).
- ▶ **Diagram:** Pyramid showing speed, capacity, and cost trade-offs.

Memory: Volatile Memory

- ▶ **Definition:** Memory that loses its contents when power is removed.
- ▶ Characteristics: High speed, limited capacity.
- ▶ Examples: SRAM, DRAM.

Memory: Persistent (Non-Volatile) Memory

- ▶ **Definition:** Memory that retains data when power is off.
- ▶ **Characteristics:** Lower speed compared to volatile memory, used for firmware and long-term storage.
- ▶ **Examples:** EEPROM, NAND Flash.

Memory: Performance Characteristics

- ▶ Key metrics: Access time, throughput, latency, bandwidth.
- ▶ Comparison: Volatile memory offers fast access; persistent memory offers long-term storage.
- ▶ Impact on system performance.

Memory: Use Cases in Embedded Systems

- ▶ Volatile Memory: For runtime data and code execution.
- ▶ Persistent Memory: For firmware, configuration data, and logging.
- ▶ Trade-offs in resource-constrained environments.

Memory: Management Techniques (Volatile)

- ▶ Memory allocation strategies: Static vs. dynamic.
- ▶ Issues: Fragmentation and allocation overhead.
- ▶ Techniques: Memory pools, manual allocation, and garbage collection (in higher-level languages).

Memory: Low-Level Memory Management

- ▶ Manual memory management in embedded systems.
- ▶ Role of compilers and linkers in memory allocation.
- ▶ Optimization strategies for performance-critical code.

Memory: Optimization Techniques

- ▶ Caching strategies and cache coherence protocols.
- ▶ Techniques such as memory prefetching and buffering.
- ▶ Efficient data structures and algorithms to minimize memory overhead.

Memory: Persistent Memory Management

- ▶ Using the Arduino `F()` macro to store constant data in flash.
- ▶ Embedded file systems (e.g., LittleFS, FAT) for persistent storage.
- ▶ Techniques for wear leveling and error correction.

Memory: Trade-Offs

- ▶ Comparison: Speed, power consumption, capacity, and cost.
- ▶ Design considerations: Reliability vs. flexibility, performance vs. persistence.
- ▶ Examples: Systems optimized for fast data access vs. systems needing robust long-term storage.

Memory: Case Studies

- ▶ Analysis of an embedded system using both volatile and persistent memory.
- ▶ Discussion of memory usage, allocation techniques, and performance outcomes.
- ▶ Real-world code examples and memory profiling.

Memory: Performance Implications

- ▶ Impact of memory speed and latency on overall system responsiveness.
- ▶ Benchmarking and performance profiling techniques.
- ▶ Strategies to minimize bottlenecks in memory access.

Memory: Future Trends

- ▶ New memory technologies: MRAM, ReRAM, 3D XPoint.
- ▶ Potential impacts on embedded system design.
- ▶ Research directions and industry trends.

Integration of Concepts

- ▶ How I/O, asynchronous programming, interrupts, and memory interact.
- ▶ **System-level architecture diagram:** CPU, memory, and I/O interconnections.
- ▶ Discussion of design trade-offs and overall system efficiency.

Case Study: Embedded System Design

- ▶ Example project using the Arduino Nano BLE 33.
- ▶ Design considerations:
 - ▶ I/O interfacing for sensor data and actuator control.
 - ▶ Asynchronous task scheduling and timer interrupts.
 - ▶ Memory management strategies for volatile and persistent storage.
- ▶ Analysis of performance, power, and reliability.

Design Trade-offs & Performance

- ▶ Balancing speed, cost, and power consumption.
- ▶ Techniques to mitigate latency and improve throughput.
- ▶ Scalability and future-proofing system designs.

Conclusion and Summary

- ▶ Recap of key topics:
 - ▶ I/O mechanisms and their role in data exchange.
 - ▶ Asynchronous programming and interrupt-driven design.
 - ▶ Memory hierarchy: Volatile vs. persistent memory and management techniques.
- ▶ These concepts underpin modern embedded and real-time systems.

Q&A and Further Reading

- ▶ Open floor for technical questions and discussion.
- ▶ **References:**
 - ▶ Patterson & Hennessy, *Computer Organization and Design*.
 - ▶ Mbed OS and Arduino documentation.
 - ▶ Relevant research papers and technical articles.
- ▶ Suggestions for further reading and exploration.